

DCLab Business Proposal for Cofounder Alignment

Prepared for: AliReza

Prepared by: DCLab founding team

Date: 18 September 2026

Purpose: Agree on what DCLab is, who it serves, what the first sellable product includes, and how we will validate the business.

Executive decision

We propose building DCLab as a working environment for data scientists and machine learning engineers. It brings the full machine learning project into one place: data, experiments, decisions, models, releases, monitoring, and the reasons behind each choice.

The first product should solve one complete customer problem well. A team should be able to bring in a business dataset, define what it wants to predict, create and compare reliable models, approve one model, produce batch predictions, and monitor the result. DCLab should guide this work with AI while keeping the user in control of important decisions.

Our initial customer is an ML team that works mainly with structured business data and currently loses time moving between notebooks, scripts, experiment tools, cloud services, documents, and manual approval steps. The strongest early use cases are customer retention, lead scoring, demand prediction, risk scoring, and similar problems where a team needs reliable batch predictions and a clear audit trail.

The immediate business goal is not to support every kind of AI project. It is to prove that a small number of real teams will use and pay for a safer, clearer, and more complete path from data to a monitored model.

DCLab in one sentence

DCLab helps an ML team move from raw business data to a verified and monitored model while keeping every experiment, decision, cost, and result connected and reviewable.

The customer problem

Most ML teams do not have one continuous system for their work. A typical project is spread across notebooks, source code, cloud jobs, spreadsheets, experiment trackers, chat messages, and deployment tools. This creates several business problems.

- Teams repeat work because they cannot easily see what was tried and why it failed.
- A result may be difficult to reproduce because the data, code, settings, and environment changed.
- Important decisions live in meetings or chat instead of the project record.

- Data leakage, weak validation, unsuitable metrics, and hidden assumptions can produce a model that looks better than it is.
- Moving a model from experimentation into business use requires handoffs between data science and engineering.
- AI assistants can generate code and suggestions, but they often lack project context and should not receive unrestricted authority.
- Cloud and AI costs can grow without clear limits or ownership.

These problems slow delivery and reduce trust. The customer needs more than an assistant that writes code. The customer needs a system that understands the project, protects the process, records the evidence, and helps the team complete the work.

The proposed product

DCLab treats the entire ML project as one connected record. Every important item has a clear version and history, including the dataset, target definition, preparation choices, features, validation method, experiments, selected model, release, predictions, monitoring results, and decisions.

The product presents the same project through three connected views.

1. **Conversation and actions** let the user ask questions, request an investigation, approve work, cancel a run, or ask why a result changed.
2. **ML workflow** shows the path from data to experiments, models, release, and monitoring.
3. **Implementation** shows the code, notebook, environment, logs, and technical evidence when the user needs them.

These are different views of the same project. A user should never have to reconcile separate versions of the truth.

A simple customer journey

Consider a company that wants to predict which customers may leave in the next 90 days.

1. The data scientist creates a DCLab project and uploads or connects the customer data.
2. DCLab checks data quality and explains missing values, imbalance, unusual patterns, and possible leakage.
3. The user confirms the business goal, prediction timing, target, review capacity, and success measure.
4. DCLab proposes an experiment plan. The user can inspect, change, approve, or reject it.
5. DCLab runs a controlled set of model candidates and records the data, code, settings, cost, and results for each run.
6. The user compares candidates using useful business tradeoffs, such as how many likely churners can be found when the retention team can review only 300 customers per week.
7. The user approves a model. DCLab packages and verifies it before use.
8. DCLab produces batch predictions for an approved dataset and records exactly which model and data created them.

9. DCLab watches for changes in the incoming data and model performance. It can recommend investigation, retraining, or rollback, but important actions remain governed by policy and approval.
- The same flow can support other structured prediction problems without changing the core product.

Who the first product serves

Primary users

Data scientists need to understand data, plan experiments, compare models, explain tradeoffs, and reproduce results.

Machine learning engineers need to automate the workflow, manage reliable jobs, control environments and cost, release models safely, monitor them, and roll them back when necessary.

Economic buyer

The likely buyer is the person responsible for ML delivery and risk: a head of data, head of machine learning, chief technology officer, or engineering leader. The buyer cares about delivery speed, repeatability, team productivity, cost control, and confidence in model releases.

Best initial customer profile

The strongest first customer is a team that:

- has at least one data scientist and one engineering owner;
- works with structured business data;
- produces recurring predictions rather than only one-off analysis;
- has working models but an unreliable path from experiment to use;
- needs to explain or reproduce decisions;
- feels the cost of disconnected tools and manual handoffs; and
- is willing to run a focused pilot on one real use case.

We should not begin with teams that require real-time global serving, large-scale video or language-model training, complex streaming systems, or a full replacement for every existing data platform.

Why customers would choose DCLab

Customer need	Common situation today	DCLab response
Complete project context	Knowledge is split across tools and people	One connected project history links data, experiments, models, decisions, and releases
Faster progress without losing control	Users manually repeat routine investigation and setup	AI proposes and explains work while controlled software performs important actions
Reliable experiments	Leakage or weak validation can make results misleading	Built-in checks protect the experiment and preserve evidence

Customer need	Common situation today	DCLab response
Reproducibility	The team cannot recreate an earlier result	Every result links to its data, code, environment, settings, and artifacts
Clear decisions	Reasons are buried in chat or meetings	Accepted and rejected decisions are stored with their evidence and rationale
Safe movement into production	Handoffs are manual and deployment state is unclear	The selected model follows a governed release, prediction, monitoring, and rollback path
Cost visibility	Compute and AI usage are spread across providers	DCLab applies budgets, quotas, estimates, and usage records to each project and run
Choice of infrastructure	Teams fear being trapped by one provider	The product can run on AWS or Google Cloud and can use approved outside compute when justified

How DCLab differs from nearby products

DCLab overlaps with several existing categories, but its product boundary is different.

Product category	What it usually does well	What DCLab adds
Notebook	Flexible exploration and code execution	A notebook becomes one governed view of a larger project rather than the project itself
Experiment tracker	Stores metrics and run details	DCLab connects experiments to business goals, approvals, releases, monitoring, and decisions
Automated ML	Trains and ranks models	DCLab makes the plan, evidence, tradeoffs, and later operational state reviewable
Coding assistant	Generates or changes code	DCLab works with project resources and requests bounded actions instead of receiving unrestricted authority
MLOps platform	Operates training and deployment infrastructure	DCLab combines operations with the user journey and durable reasons behind each decision

We should integrate proven tools for specialized functions where they help. DCLab's value comes from owning the connected customer workflow, the project history, and the rules for safe action.

The first sellable product

The MVP should support one complete path from data to a monitored batch model.

Included in the MVP

- secure workspaces, projects, members, and roles;
- data upload and a small set of useful data connections;
- data profiling, quality checks, leakage checks, and clear findings;
- definition of the prediction goal and business constraints;
- reviewable experiment plans and controlled model runs;
- model comparison with business-relevant metrics and cost;
- a project assistant that can explain, investigate, and propose changes;
- a controlled improvement loop with budgets and stop conditions;
- project history that records important accepted and rejected decisions;
- a managed notebook for deeper analysis, with isolated code execution only after its security tests pass;
- Python tools and command-line access for ML engineers;
- a verified model package, batch predictions, monitoring, and rollback;
- deployment on either AWS or Google Cloud; and
- optional outside CPU or GPU resources when policy, data sensitivity, availability, and cost permit.

Excluded from the MVP

- unrestricted autonomous changes;
- real-time model serving as the main release path;
- autonomous retraining without review;
- every possible database, SaaS, and data connector;
- large distributed foundation-model training;
- active operation of the same customer database across two clouds;
- a marketplace of third-party agents; and
- replacing a customer's entire data warehouse, source-control system, or business application.

These boundaries keep the first promise understandable and testable.

Current position

DCLab is not starting from a slide deck. The repository already contains a meaningful deterministic ML foundation: project and workspace structures, data and model records, repeatable training behavior, scientific checks, stored artifacts, background jobs, reports, predictions, and broad automated testing.

The complete product experience is not finished. The unified lifecycle view, durable project decision memory, full agent system, managed notebook runtime, public developer interfaces, production connector set, monitored model-release path, and production cloud proof remain in the implementation program.

This distinction matters when we speak with customers. We can demonstrate the foundation and design direction, but we should label planned capabilities honestly until they pass their release tests.

Area	Current position	Business implication
Deterministic ML engine	Meaningful foundation exists	We can build on tested scientific behavior instead of starting with an AI chat shell
Product workflow	Partially available and still fragmented	The next priority is one complete customer journey
AI and agents	Narrow AI support exists; governed agent system is planned	Early demos must not imply unrestricted autonomy
Notebook and external compute	Detailed design and work packages exist	These should be enabled only after security and cost controls pass
Production operations	AWS and Google Cloud architecture is defined; production proof remains	Pilot commitments must match verified deployment readiness

Technical side in plain language

DCLab needs two connected technical systems. The first is the **platform**, which stores the customer account, project history, permissions, decisions, costs, and current model status. The second is the **runtime**, which performs temporary work such as notebook code, data checks, model training, and prediction. Keeping these systems separate lets DCLab stop or replace a compute job without losing the project record.

The platform should run as one complete installation on either AWS or Google Cloud. Each installation has one authoritative database and one authoritative object store. DCLab may send an approved job to outside CPU or GPU capacity, but Runpod, Railway, Lambda GPU Cloud, Vast.ai, and Nebius remain optional execution providers. They never become the owner of customer identity, project state, secrets, approvals, or audit history.

How the technical system works

Layer	Responsibility	Why it matters to the business
Web product and developer access	Give users one interface, API, Python SDK, command line, and controlled agent integration	Different users can work with the same project without creating separate versions of the truth
DCLab platform services	Check identity and permissions, enforce policy, coordinate work, record decisions, and expose stable product actions	Customer rules remain consistent whether work starts from the interface, notebook, or automation
PostgreSQL database	Store accounts, workspaces, projects, versions, job state, approvals, usage, and audit records	The company can explain who did what, when it happened, and which version is current
Object storage	Store large datasets, model packages, reports, logs, and prediction files with verified identifiers	Large files remain durable and reproducible without overloading the main database

Layer	Responsibility	Why it matters to the business
Job and runtime system	Run profiling, training, notebook sessions, evaluation, packaging, and batch prediction in isolated workers	Expensive or risky work can be queued, limited, cancelled, retried, and scaled independently
ML evidence services	Record experiment details, validate data and model inputs, and calculate monitoring results behind DCLab services	Teams receive specialist ML evidence without creating a second product authority
Cloud operations	Provide networking, encryption, secret management, backups, monitoring, alerts, and recovery on AWS or Google Cloud	The same DCLab release can meet production requirements in the customer or company cloud of choice

PostgreSQL stores product truth. Private object storage stores large artifacts. A durable job system coordinates work. MLflow may record detailed experiment and model-package metadata, but DCLab owns model approval, release, rollback, tenancy, and audit. Jupyter provides the familiar code-cell experience, while DCLab owns session authorization, limits, lifecycle, and evidence.

Technical MVP

The technical MVP must support the first sellable journey safely from beginning to end. A feature is included only when it is usable through the product, authorized for the correct workspace, observable in production, and covered by recovery and failure tests.

Capability	MVP commitment
Identity and tenancy	Secure browser sessions, API credentials for non-browser clients, workspace roles, project permissions, audit records, and strict separation between customers
Product data	PostgreSQL as the single source of lifecycle truth, versioned database migrations, tested backups, point-in-time recovery, and explicit retention and deletion behavior
Files and artifacts	Private S3 on AWS or Cloud Storage on Google Cloud, immutable versions and digests, encryption, lifecycle rules, and authorized download or upload paths
Data access	Direct CSV and Parquet upload, AWS S3 and Google Cloud Storage objects, read-only SQL or PostgreSQL, one CRM source, and read-only Snowflake through one connector contract
ML workflow	Deterministic profiling, quality and leakage checks, target definition, validation planning, controlled training, model comparison, model packaging, batch prediction, drift monitoring, and rollback

Capability	MVP commitment
Experiment evidence	One private MLflow service for detailed run and package metadata, reconciled with DCLab records and never exposed as a second customer control plane
Notebook runtime	Isolated Python notebook sessions using Jupyter Server first, with explicit start, stop, idle timeout, absolute expiry, quotas, network policy, file limits, cancellation, and cleanup
Agent behavior	Project-aware explanation and proposals, approved read tools, bounded actions through the same services used by the product, budgets, stop conditions, and human approval for material changes
Jobs and recovery	Durable queues, idempotent commands, retry limits, cancellation, heartbeats, stale-job recovery, late-result rejection, and visible failure reasons
Developer access	Versioned API, Python SDK, command-line tools, and Model Context Protocol access that follow the same permissions and action contracts as the web product
Deployment	One portable application and Kubernetes workload contract, deployed independently to either AWS or Google Cloud with short-lived workload identity and no static cloud keys
Production operations	Central logs, metrics, traces, service targets, alerts, runbooks, cost labels, budgets, security checks, disaster recovery, and a tested rollback path

The notebook starts with Jupyter Server in an isolated runtime for each authorized session. Jupyter Enterprise Gateway is deferred until DCLab needs shared multi-cluster kernel routing and its security and lifecycle behavior passes the same tests. This keeps the first implementation smaller while preserving a clear path to larger runtime fleets.

The MVP should prove home-cloud CPU execution and one external GPU provider before claiming a broad compute marketplace. Runpod is a reasonable first external GPU candidate. Railway, Lambda GPU Cloud, Vast.ai, and Nebius can be added through the same provider contract after their capacity, isolation, networking, data-transfer, cleanup, and cost behavior is verified. A provider can be enabled for training without automatically being trusted for notebooks or model serving.

Technical capabilities after the MVP

After the first customer journey is proven, the technical roadmap can expand in response to paid demand. These items are directions, not launch commitments.

Area	After the MVP
Prediction delivery	Add low-latency online endpoints, scheduled prediction services, streaming inputs, autoscaling, and stronger release traffic controls
Model development	Add more model families, distributed training, deeper GPU support, feature reuse, advanced tuning, and larger datasets where customers demonstrate demand
Notebook scale	Add Jupyter Enterprise Gateway or an equivalent remote-kernel control layer when multi-cluster scheduling provides clear operational value
Compute network	Certify additional Runpod, Railway, Lambda GPU Cloud, Vast.ai, and Nebius workload types, regions, accelerators, and reserved-capacity options
Data connections	Add more warehouses, databases, SaaS systems, event sources, and customer-managed network connections
Automation	Allow policy-approved retraining, evaluation, release preparation, and rollback recommendations with stronger simulation, evaluation, and approval controls
Enterprise controls	Add single sign-on, automated user provisioning, customer-managed encryption keys, private networking, regional controls, compliance evidence, and support commitments
Reliability and scale	Add multi-region recovery, larger workload fleets, advanced scheduling, capacity reservations, and stricter service targets where contracts require them
Ecosystem	Add reviewed extensions, customer tools, and partner integrations without allowing them to bypass DCLab permissions, evidence, or audit rules

Technical rules that do not change

- DCLab services, not an agent, notebook, MLflow, or compute provider, decide whether an action is allowed.
- Models and agents do not receive direct database credentials or unrestricted access to customer secrets.
- Every important dataset, environment, experiment, model, and prediction output has an immutable version or verified digest.
- Jobs must be cancellable, retry-safe, bounded by time and cost, and recoverable after a worker failure.
- Important actions remain visible, attributable, reversible where possible, and linked to the evidence used for the decision.
- A capability is advertised only after its security, failure, recovery, and production behavior has been tested in the supported deployment.

Business model hypothesis

We should test a subscription plus usage model.

Platform subscription would cover the workspace, collaboration, project history, governance, standard integrations, and product support. Pricing can grow by team size, active projects, or an agreed workspace level.

Usage charges would cover variable costs such as model training, notebook sessions, storage, batch prediction, and AI model usage. Customers should see estimates, budgets, and actual usage rather than receive an unexplained bill.

Enterprise terms could later cover private deployment, advanced identity and security controls, support commitments, audit requirements, and custom integrations.

We should not set final prices before pilot customers show which outcome they value most and how much compute a normal project consumes. The first commercial offer can be a paid, time-boxed design-partner pilot with a defined use case, support boundary, success measures, and conversion option.

Initial route to market

The first sales motion should be founder-led and narrow.

1. Select three to five design partners with a real structured-data prediction problem.
2. Qualify each partner for data access, a named data scientist, an engineering owner, and executive support.
3. Choose one use case per partner and agree on the baseline process before DCLab.
4. Run the complete DCLab journey with clear product boundaries and weekly feedback.
5. Measure time to a verified model, reproducibility, number of manual handoffs, user confidence, usage cost, and willingness to continue.
6. Convert successful pilots into annual subscriptions with usage terms.

Useful entry points include customer retention, lead prioritization, credit or fraud review support, demand planning, and operational risk. We should choose the first vertical based on access to design partners and the speed at which we can obtain safe, representative data.

Measures of success

The MVP should be judged by customer outcomes rather than the number of features.

Product proof

- A data scientist completes the supported journey without manual database work.
- An ML engineer reproduces the result and runs the supported workflow through developer tools.
- A second authorized user can explain which data, settings, code, and decisions produced the selected model.
- The team can cancel work, reject a proposal, roll back a release, and see the full history.
- Every released result is linked to its evidence and passes the required scientific and security checks.

Business proof

- Design partners use DCLab on real recurring work rather than a demonstration dataset alone.
- Users return to the same project and continue from its recorded history.
- The product reduces a measurable delay, repeated task, or handoff in the customer's current process.
- At least some pilot customers are willing to pay for continued use.
- Revenue covers a credible share of compute and AI costs, with a clear path to healthy gross margin.

We should set numeric targets with each design partner after measuring its current process. Inventing universal speed or cost claims before those measurements would weaken the proposal.

Delivery approach

Stage one trustworthy foundation

Close the remaining identity, data policy, interface consistency, and verification gaps. Preserve the reliable ML behavior that already exists.

Stage two complete core journey

Deliver the connected path from dataset understanding through experiment comparison and model selection. Make the workflow usable before expanding autonomy.

Stage three controlled assistance

Add project-aware investigation, proposals, decision memory, developer tools, and the bounded improvement loop. Important changes remain visible, reviewable, and reversible.

Stage four production pilot

Complete the isolated notebook, connectors, batch release, monitoring, rollback, cloud deployment, support process, and pilot evidence. Only verified capabilities appear in customer commitments.

The detailed engineering roadmap remains much larger than this business summary. We should manage it as small, reviewable work packages while keeping the customer journey as the main priority.

Main risks and responses

Risk	Why it matters	Proposed response
Scope becomes too large	The roadmap includes many valuable capabilities	Protect the one data-to-batch-model journey and delay features that do not unblock it
Customers do not trust AI actions	A wrong change can damage a model or release	Start with explanation and proposals, require approval for risk, and keep rollback available

Risk	Why it matters	Proposed response
Product feels like many tools joined together	Users may return to their existing stack	Use one project record, shared identifiers, and connected views across the full journey
Infrastructure cost becomes unpredictable	GPU and AI usage can damage margins	Use budgets, quotas, estimates, provider controls, and per-project usage records
Cloud or vendor dependence limits adoption	Customers have different security and procurement needs	Keep the product portable between AWS and Google Cloud and place optional compute behind controlled adapters
Security work delays the notebook	User code is a high-risk capability	Keep managed analysis available and leave arbitrary code disabled until isolation passes
We build before confirming demand	Technical progress can hide weak customer pull	Run founder-led design partnerships and tie roadmap priority to observed customer work

Defensibility we intend to build

DCLab's long-term defensibility should come from the product system and accumulated customer context rather than a single model or provider.

- The connected lifecycle makes the effect of a data or model change visible across the project.
- Decision history preserves why a team accepted, rejected, or replaced an approach.
- The policy and evaluation layer allows AI assistance to grow without giving away uncontrolled authority.
- Reusable project patterns and evidence can improve future work while remaining within customer permissions.
- The shared workflow across the interface, notebook, developer tools, and cloud jobs creates switching value once a team relies on it for real projects.

These advantages must be earned through product use. They should be treated as a direction to validate, not as a claim that the business already has a moat.

Cofounder decisions

AliReza, the next step is to agree on the following points before we broaden implementation or make external promises.

1. **Product definition** Approve DCLab as the project environment for the connected ML lifecycle, with AI assistance governed by DCLab.
2. **First customer** Approve structured-data ML teams with recurring batch prediction needs as the initial segment.
3. **MVP promise** Approve one complete journey from data to a verified, monitored batch model.

4. **Commercial test** Approve founder-led design partnerships and a paid pilot structure before final pricing.
5. **Product boundaries** Agree that real-time serving, fully autonomous retraining, broad connector coverage, and large-scale foundation-model training are outside the first release.
6. **Ownership** Assign one founder to customer discovery and pilot development, one to product and engineering delivery, and review evidence together each week. The exact split should follow our strengths and availability.

Proposed agreement

If we agree with this proposal, we will use it as the plain-language business definition of DCLab. The technical roadmap will remain the implementation authority, but every near-term engineering priority must support the approved MVP journey, a pilot requirement, or a mandatory safety condition.

Our next business milestone is a design partner completing one real project and choosing to continue because DCLab made the work clearer, more repeatable, and easier to operate. That result will tell us more than adding another disconnected feature.